Paradigmi di Programmazione

Diego Stefanini — prof.ssa Chiara Bodei, a.a. 2026-27

Indice

1 Il lambda calcolo 3
1.1 Calcolabilità e paradigmi 3
1.2 Sintassi dei λ-termini 5
1.3 Variabili libere e legate 6
1.4 Sostituzione e β-riduzione 8
1.5 Ordine di riduzione e strategie 10
1.6 Funzioni con più argomenti 13
1.7 Ricorsione: il combinatore Y 14
1.8 Codifiche: booleani e numeri 15
2 Controllo dei tipi 18
2.1 Perché servono i tipi 18
2.2 Sistemi di tipi 19
2.3 Regole di inferenza 20
2.4 Un linguaggio di espressioni 21
2.5 Type safety 22
3 Il lambda calcolo tipato 24
3.1 Tipi e regole di tipo 24
3.2 Type safety nel λ-calcolo tipato 25
3.3 Estensioni e ricorsione 26
3.4 Cosa possono garantire i tipi 27

1 Il lambda calcolo

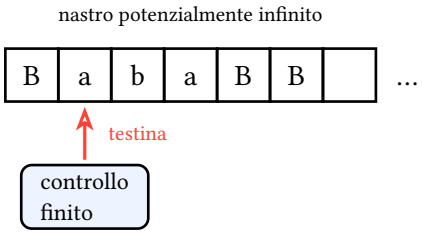
1.1 Calcolabilità e paradigmi

Hilbert, Entscheidungsproblem (problema della decisione): esiste una procedura **del tutto meccanica** che, data una qualunque formula della logica del primo ordine (un'affermazione matematica scritta in un linguaggio formale), dice se è un teorema, cioè se si può dimostrare?

Per rispondere bisogna prima dire cos'è una «procedura meccanica», cioè un **algoritmo**. Tre risposte diverse, quasi insieme:

lambda calcolo	**funzioni ricorsive**	**macchina di Turing**
Alonzo Church, 1935	Kurt Gödel, Stephen Kleene, 1935	Alan Turing, 1936

Com'è fatta la **macchina di Turing**:



- Macchina **ideale**, non fisica: legge e scrive simboli su un nastro, secondo regole fissate.
- Esempio di regola: *legge a, riscrive a e si sposta a destra.*
- È il modello astratto di una macchina che esegue algoritmi.

Macchina di Turing Universale (UTM): una macchina di Turing che, data la **descrizione** di un'altra macchina e i suoi dati, la simula. È il modello teorico del computer: il programma è un dato.

Con la UTM Turing risponde **no** all'Entscheidungsproblem: esistono problemi che nessuna macchina di calcolo può risolvere.

I tre formalismi, anche se sono fatti in modo diversissimo, calcolano esattamente le stesse funzioni. Da qui la **tesi di Church-Turing**. Nella figura, i modelli meno potenti sono quelli che calcolano solo **funzioni totali** (danno sempre un risultato, per ogni input) e, ancora più dentro, le espressioni regolari:



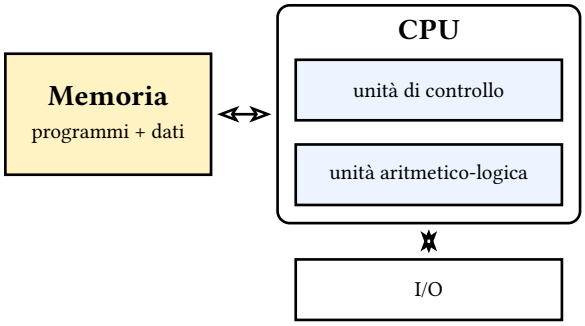
più è interno, meno è potente

Se una funzione è calcolabile con un qualunque formalismo, allora esiste una macchina di Turing che la calcola.

È **indimostrabile** (i formalismi possibili sono infiniti) ma universalmente accettata: non si conosce nessun modello più potente.

Un linguaggio è **Turing completo** se calcola tutto ciò che calcola una macchina di Turing. Si dimostra scrivendo nel linguaggio un programma che **simula** una macchina di Turing.

Dai due modelli, Turing e Church, nascono due famiglie di linguaggi. Da Turing vengono i linguaggi **imperativi**, passando per la macchina di von Neumann:



Architettura di von Neumann, ispirata alla macchina di Turing ma concreta:

- **memoria** con programmi e dati;
- **CPU** che preleva un'istruzione alla volta, la interpreta e la esegue.

La generalità viene dal **programma memorizzato**: il programma sta in memoria come un dato qualsiasi, quindi la stessa macchina esegue qualunque programma basta caricarlo.

Un linguaggio «alla von Neumann» riproduce ad alto livello questa struttura:

Nel linguaggio		**Nella macchina**
variabili	↔	celle di memoria (il nastro)
istruzioni di controllo (if, cicli)	↔	test & jump (controllo una condizione e salto)
assegnamento	↔	modifica dello stato (leggo e scrivo la memoria)

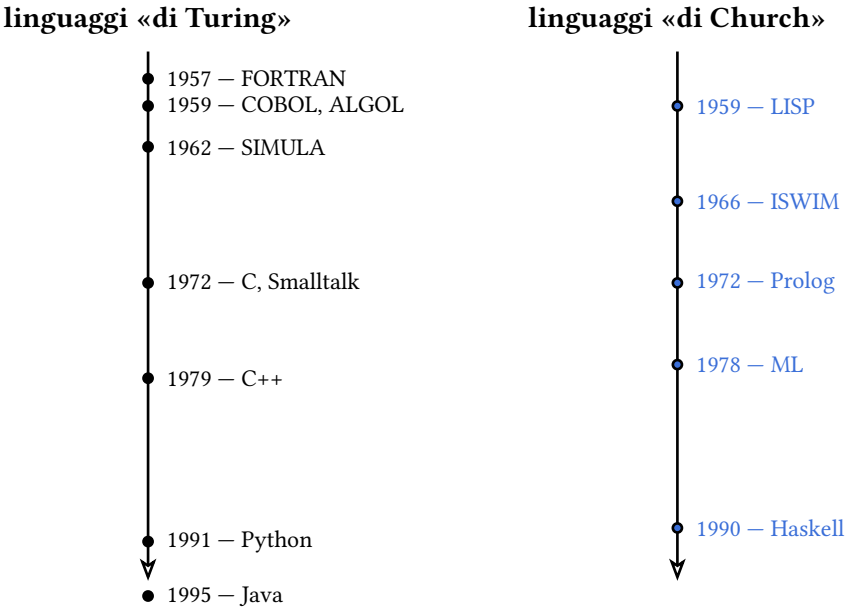
Backus: l'assegnamento divide la programmazione in due mondi.

espressioni
spazio matematico ordinato, con proprietà algebriche utili. Lì avviene la maggior parte del calcolo

istruzioni
spazio disordinato, con poche proprietà utili. La programmazione strutturata prova a metterci un po' d'ordine

Da Church vengono invece i linguaggi **funzionali**.

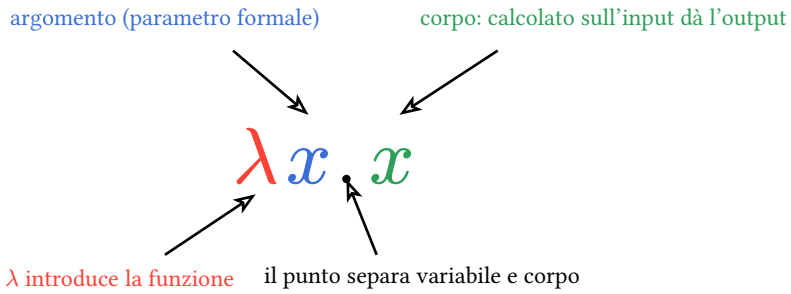
Programmazione funzionale: il programma è una serie di **valutazioni di funzioni matematiche**. Punto di forza: niente **effetti collaterali**. Una funzione ha un effetto collaterale quando, oltre a restituire un valore, cambia qualcosa fuori da sé (una variabile globale, un file, lo schermo). Senza effetti collaterali una funzione con lo stesso input dà sempre lo stesso output, quindi è più facile verificare che il programma sia corretto e ottimizzarlo. Il λ-calcolo (Church, 1935) è il primo linguaggio funzionale.



Le due linee di discendenza

1.2 Sintassi dei λ-termini

Il λ-calcolo ha solo due operazioni. La prima è l'**astrazione**, cioè definire una funzione:



La **legge di corrispondenza** dice cosa fa la funzione su ogni input: qui $\forall x.f(x) = x$, cioè a ogni x associa x stesso. È la **funzione identità**.

La seconda è l'**applicazione**. Applicare una funzione = darle un **parametro attuale** al posto del parametro formale. Immagina la funzione come una scatola:

$$z \;\to\; \boxed{\lambda x.x} \;\to\; z \qquad \textbf{identità}\text{: restituisce l'input così com'è}$$

$$z \;\to\; \boxed{\lambda x.y} \;\to\; y \qquad \textbf{costante}\text{: restituisce sempre } y \;\; (\forall x.f(x) = y)$$

$$z, w \;\to\; \boxed{\lambda x.\lambda y.x} \;\to\; z \qquad \textbf{selezione}\text{: prende due argomenti e restituisce il primo}$$

La selezione passo per passo: $((\lambda x.\lambda y.x)z)w \to (\lambda y.z)w \to z$. Nel primo passo metto z al posto di x nel corpo $\lambda y.x$; nel secondo metto w al posto di y nel corpo z, ma y non c'è, quindi w sparisce e resta z.

Queste due operazioni, più le variabili, sono tutto il linguaggio. Un programma è un'espressione (**λ-espressione**). Ci sono solo tre modi di costruirla:

e	::= x	variabile
	$\mid \lambda x.e$	astrazione funzionale (dichiarazione di funzione)
	$\mid e\ e$	applicazione (chiamata di funzione)

Si legge: un'espressione e è una variabile, **oppure** (|) un'astrazione, **oppure** un'applicazione. La definizione è **ricorsiva**: dentro $\lambda x.e$ e dentro $e\ e$ ci sono altre espressioni, costruite con le stesse tre regole. Per esempio $(\lambda x.x)y$ è un'applicazione di $\lambda x.x$ (astrazione, con corpo la variabile x) alla variabile y.

$\lambda x.e$ è una **funzione anonima**: non ha nome, e x è la dichiarazione del suo parametro. Lo stesso concetto nei linguaggi:

Linguaggio	$x \mapsto x + 1$
JavaScript	`function(a){ return a + 1; }` oppure `a => a+1`
OCaml	`fun x -> x+1`
Java (si chiamano lambda)	`(int x) -> x + 1`

Da qui in poi useremo lettere come e, e_1, e_2, e_3. **Non** sono variabili del λ-calcolo: sono nomi che stanno per **un'espressione qualsiasi**, come in algebra $a + b$ vale per due numeri qualsiasi.

In $e_1 e_2$, e_1 è la funzione che chiamo ed e_2 l'argomento che le passo (in JavaScript `e1(e2)`). La funzione non deve per forza avere un nome: la sua definizione può stare **direttamente dentro** la chiamata.

$$\underbrace{(\lambda x.(\lambda y.xy))}_{e_1 = \text{ la funzione}} \quad \underbrace{(\lambda z.z)}_{e_2 = \text{ l'argomento}}$$

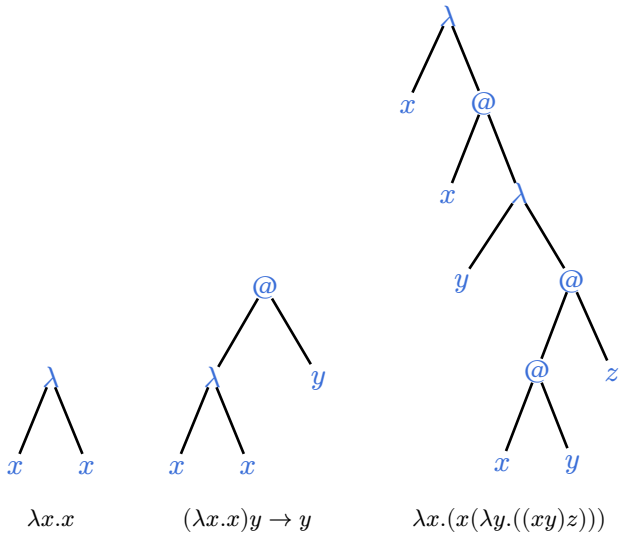
Senza parentesi $\lambda x.xy$ è ambiguo: è $\lambda x.(xy)$ (prende x e restituisce x applicata a y) oppure $(\lambda x.x)y$ (l'identità applicata a y)? Decidono due convenzioni (importanti), che servono anche a scrivere meno parentesi:

1. Lo scope di λ va il più a destra possibile
Il corpo di una λ è **tutto** quello che c'è dopo il punto, fino alla fine (o fino a una parentesi chiusa).
$\lambda x.xy$ è $\lambda x.(xy)$
$\lambda x.\lambda y.xy$ è $\lambda x.(\lambda y.(xy))$

2. L'applicazione associa a sinistra
Con più applicazioni di fila si parte da sinistra: prima e_1 applicata a e_2, poi il risultato applicato a e_3. Come `f(a)(b)` in JavaScript.
$e_1 e_2 e_3$ è $(e_1 e_2) e_3$

Esercizio: quali parentesi sono sottintese in $\lambda x.x\lambda y.xyz$?
1. Regola 1. Il corpo di λx è tutto il resto, $x\lambda y.xyz$; dentro, il corpo di λy è xyz: $\lambda x.(x\lambda y.(xyz))$
2. Regola 2. xyz diventa $(xy)z$; e $x\lambda y.\ldots$ è x applicata alla funzione $\lambda y.\ldots$: $\lambda x.(x(\lambda y.((xy)z)))$

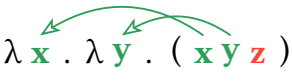
Ogni termine si può disegnare come un **albero**, che segue l'annidamento completo delle parentesi. Serve a vedere quali passi di valutazione si possono fare. Nodo λ: figlio sinistro il parametro, destro il corpo. Nodo @ (applicazione): figli funzione e argomento.



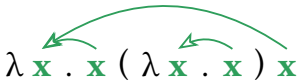
1.3 Variabili libere e legate

λ è un **operatore di binding**: in $\lambda x.e$ lega la x dentro e (il suo **scope**), come $\forall x$ in $\forall x.P$ nella logica. È l'**unico** modo di associare valori a variabili nel λ-calcolo.

- **Legata**: introdotta da un λ che la contiene. **verde**, la freccia punta al suo λ.
- **Libera**: nessun λ la dichiara. **rossa**.

Espressione	Chi è legato
	x legata
	x e y legate, z libera
	f legata, x e y libere
	la prima f è legata; x e **la seconda f sono libere**: sta fuori dalle parentesi, fuori dallo scope di λf

Una stessa variabile può voler dire cose diverse in punti diversi:



che con le parentesi esplicite è $\lambda x.(x(\lambda x.x)x)$

La x nel λx interno è legata a **quel** λ, non a quello esterno. Tre x, due significati: da qui nasce il conflitto di nomi.

Applicando senza fare attenzione ai nomi nasce un problema:

	Espressione	Risultato
ingenuo	$(\lambda x.\lambda y.xy)y$	$\lambda y.yy$ ✗ sbagliato
rinomino y in z	$(\lambda x.\lambda z.xz)y$	$\lambda z.yz$ ✓

Le due espressioni di partenza sono **la stessa funzione** (ho solo cambiato il nome del parametro), ma i risultati si comportano in modo diverso. Nel primo caso la y libera che passo come argomento finisce **catturata** dal λy interno: è un **conflitto di nomi**.

x in $\lambda x.e$ è un segnaposto: si può rinominare con una variabile **fresca**, cioè un nome che non compare da nessuna parte nelle espressioni che stiamo trattando. È così che si risolve il conflitto di nomi, come nella riga «rinomino» della tabella.

Le variabili libere si possono definire in modo preciso, con una **definizione induttiva**: un caso per ogni elemento della sintassi.

$$\begin{aligned} \mathrm{FV}(x) &= \{x\} && \text{variabile} \\ \mathrm{FV}(e_1 e_2) &= \mathrm{FV}(e_1) \cup \mathrm{FV}(e_2) && \text{applicazione: le libere di tutte e due} \\ \mathrm{FV}(\lambda x.e) &= \mathrm{FV}(e) \setminus \{x\} && \text{astrazione: il } \lambda \text{ lega } x\text{, la tolgo} \end{aligned}$$

FV sta per *Free Variables*. Una variabile che non è libera è **legata** (*bound*). Un termine senza variabili libere ($\mathrm{FV}(e) = \varnothing$) si dice **chiuso**; i termini chiusi si chiamano **combinatori**, il più semplice è l'identità $\lambda x.x$.

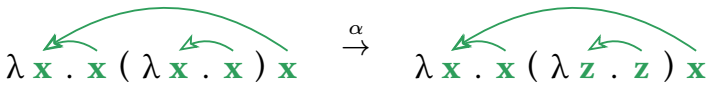
Esercizio: $\mathrm{FV}((\lambda x.\lambda y.xy)((\lambda z.z)k))$

$$= \mathrm{FV}(\lambda x.\lambda y.xy) \cup \mathrm{FV}((\lambda z.z)k) = \varnothing \cup \{k\} = \{k\}$$

Cambiare nome a un parametro non cambia niente: $\lambda a.ac$ e $\lambda b.bc$ si dicono **α-equivalenti**, perché a e b contano solo per il **ruolo** che hanno nell'espressione. Lo stesso vale nei linguaggi:

`function(a){ return a + 1; }` è α-equivalente a `function(b){ return b + 1; }`

Espressioni α-equivalenti rappresentano **lo stesso programma**. Rinominare una variabile legata con una variabile fresca si chiama **α-conversione**. Serve a passare da $\lambda x.x$ a $\lambda z.z$, e soprattutto a togliere il conflitto di nomi visto sopra (*variable shadowing*):



i due termini sono equivalenti a meno di α-conversione, ma nel secondo ogni nome ha un solo significato

1.4 Sostituzione e β-riduzione

Cosa vuol dire **eseguire** (valutare) una λ-espressione? La valutazione, *eval*, fa solo una cosa: **chiamare funzioni**.

eval$((\lambda x.e_1)e_2)$: rimpiazzo ogni occorrenza di x in e_1 con e_2, poi valuto il termine che ne risulta.

La **sostituzione** si scrive $e_1\{x := e_2\}$ (oppure $e_1\{e_2/x\}$): è e_1 con ogni occorrenza **libera** di x sostituita da e_2. Per esempio $xz\{x := \lambda y.y\} = (\lambda y.y)z$.

Il problema: e se e_2 contiene una variabile libera che in e_1 è legata? Sostituendo alla cieca finisce **catturata** dal λ di e_1, e legare una variabile libera cambia il significato. Esempio con le operazioni aritmetiche, per semplicità: $(\lambda x.(x * y))\{y := (x + x)\}$.

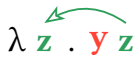
Come	**Risultato**
alla cieca	$\lambda x.(x * (x + x))$ ✗ le x di $x + x$ ora sono il parametro
α-conversione, z fresca	$(\lambda z.(z * y))\{y := (x + x)\} = \lambda z.(z * (x + x))$ ✓

La soluzione è la **sostituzione che evita la cattura** (*capture-avoiding substitution*): prima di sostituire, se serve, rinomino con l'α-conversione. La definizione ha un caso per ogni elemento della sintassi:

	Regola	**Perché**
variabile	$x\{x := e\} \equiv e$	è proprio la variabile da sostituire
	$y\{x := e\} \equiv y$ se $x \neq y$	un'altra variabile non si tocca
applicazione	$(e_1e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$	sostituisco da tutte e due le parti
astrazione	$(\lambda x.e_1)\{x := e\} \equiv \lambda x.e_1$	qui x è legata: la sostituzione vale solo per le variabili libere, quindi non cambia niente
	$(\lambda y.e_1)\{x := e\} \equiv \lambda y.(e_1\{x := e\})$ se $y \neq x$ e $y \notin \mathrm{FV}(e)$	in e non compare y: nessun rischio di conflitto
	$(\lambda y.e_1)\{x := e\} \equiv \lambda z.((e_1\{y := z\})\{x := e\})$ se $y \neq x$ e $y \in \mathrm{FV}(e)$, z fresca	la y libera di e verrebbe catturata: prima α-converto y in z, poi sostituisco

«Fresca» qui vuol dire che non compare né in e_1 né in e. L'ultimo caso, disegnato: sostituire x con y nel corpo di $\lambda x.\lambda y.xy$.





alla cieca: la y arriva in un mondo dove esiste già una y legata e diventa indistinguibile da lei ✗	prima rinomino y in z: ora la y resta libera ✓

Un esempio completo: $(\lambda x.\lambda y.((\lambda z.z)x))y$. Il passo $(\lambda y.((\lambda z.z)x))\{x := y\}$ fatto alla cieca **non va bene**: le due y sono diverse ($y \in \mathrm{FV}(e)$). Quindi, con k fresca:

$$(\lambda y.((\lambda z.z)x))\{x := y\} \equiv_\alpha (\lambda k.((\lambda z.z)x))\{x := y\} = \lambda k.((\lambda z.z)y)$$

Esercizi. L'espressione è sempre la stessa, cambia la variabile da sostituire ($\equiv_\alpha$ vuol dire «α-equivalente», cioè uguale a meno di rinominare):

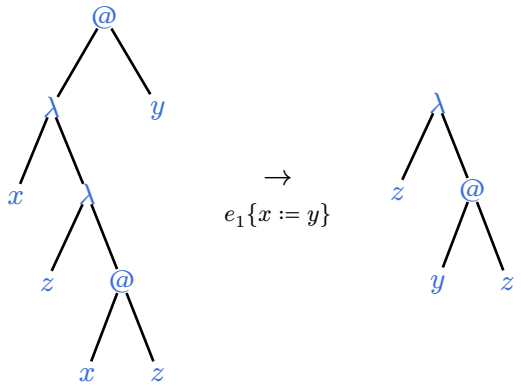
Sostituzione	**Risultato**
$((\lambda x.yx)w)\{x := \lambda k.kx\}$	$(\lambda x.yx)w$ l'unica x è legata: $(\lambda x.e_1)\{x := e\} \equiv \lambda x.e_1$
$((\lambda x.yx)w)\{y := \lambda k.kx\}$	$\equiv_\alpha ((\lambda z.yz)w)\{y := \lambda k.kx\} = (\lambda z.(\lambda k.kx)z)w$ la x di $\lambda k.kx$ è libera e verrebbe catturata da λx: rinomino
$((\lambda x.yx)w)\{w := \lambda k.kx\}$	$(\lambda x.yx)(\lambda k.kx)$ w non è sotto nessun λ: caso applicazione

Con la sostituzione si scrive la regola fondamentale del λ-calcolo, la **β-riduzione**:

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\}$$

- Cattura esattamente l'**applicazione di funzione**: $(\lambda x.x)3 \to 3$.
- Un **redex** (espressione riducibile) è una sottoespressione della forma $(\lambda x.e_1)e_2$, a cui la regola si può applicare.

Esempio: $(\lambda x.\lambda z.xz)y$. Il parametro formale è x, il corpo è $e_1 = \lambda z.xz$, il parametro attuale è $e_2 = y$:



$$(\lambda x.\lambda z.xz)y \to \lambda z.yz$$

Il λ-calcolo è di **ordine superiore**: una funzione può prendere funzioni come parametri e restituire funzioni come risultato, in modo naturale.

Espressione	**Cosa fa**
$\lambda x.x$	identità
$\lambda y.(\lambda x.x)$	scarta l'argomento y e restituisce l'identità: è una **funzione costante**
$\lambda f.f(\lambda x.x)$	data una funzione f, la **invoca sull'identità**

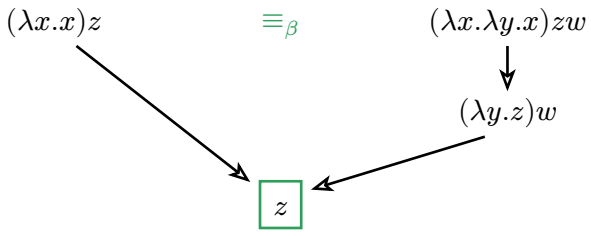
Applicazione	Passi
$(\lambda x.x)y$	$\to y$
$(\lambda x.x)(\lambda y.y)$	$\to \lambda y.y$ l'identità applicata all'identità
$(\lambda x.xy)z$	$\to zy$ prende una funzione z e la applica a y
$(\lambda x.xy)(\lambda z.z)$	$\to (\lambda z.z)y \to y$ il parametro attuale è l'identità
$(\lambda x.((\lambda y.y)x))z$	$\to (\lambda y.y)z \to z$
$(\lambda f.fz)(\lambda x.x)$	$\to (\lambda x.x)z \to z$ prende una funzione e la applica a z
$(\lambda x.(xx))(\lambda y.y)$	$\to (\lambda y.y)(\lambda y.y) \to \lambda y.y$ due termini uguali con ruoli diversi: funzione e argomento
$((\lambda x.\lambda y.x + y)3)5$	$\to (\lambda y.3 + y)5 \to 3 + 5 \to 8$ (supponendo di avere il +)
$((\lambda x.\lambda y.xy)(\lambda x.x))z$	$\to (\lambda y.(\lambda x.x)y)z \to (\lambda x.x)z \to z$ forma $e_1e_2e_3$
$(\lambda x.\lambda y.xy)zk$	$\to (\lambda y.zy)k \to zk$

La valutazione va avanti scegliendo un redex e riducendolo. Quando non ci sono più redex l'espressione è in **forma normale β**: non si può più riscrivere con la β-riduzione, ed è il **risultato finale**, il **valore calcolato**. Per esempio $\lambda x.x$ e $\lambda t.\lambda f.t$ sono valori: **le funzioni sono valori**.

Notazione	Significato
$e_1 \to e_2$	e_2 si ottiene da e_1 con **un** passo di riduzione
$e_1 \Rightarrow e_2$	e_2 si ottiene da e_1 con **zero o più** passi

Un passo di β-riduzione è un passo di calcolo, quindi $\Rightarrow$ (la **chiusura riflessiva e transitiva** di $\to$) rappresenta una computazione qualsiasi. Quando $e_1 \Rightarrow e_2$ si dice che e_1 è **β-riducibile** a e_2.

Con $\Rightarrow$ si definisce quando due espressioni sono «uguali». e_1 ed e_2 sono **β-equivalenti**, $e_1 \equiv_\beta e_2$, se:
1. sono identiche a meno di α-conversione, oppure
2. $e_1 \Rightarrow e_2$ oppure $e_2 \Rightarrow e_1$, oppure
3. $e_1 \Rightarrow e$ e anche $e_2 \Rightarrow e$ (arrivano alla stessa espressione).

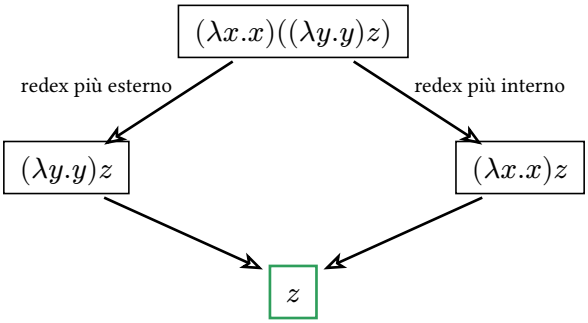


β-equivalenti per il terzo caso: tutte e due si riducono a z

Intuizione: due espressioni sono β-equivalenti quando sono indistinguibili dal punto di vista del calcolo, cioè calcolano gli stessi risultati.

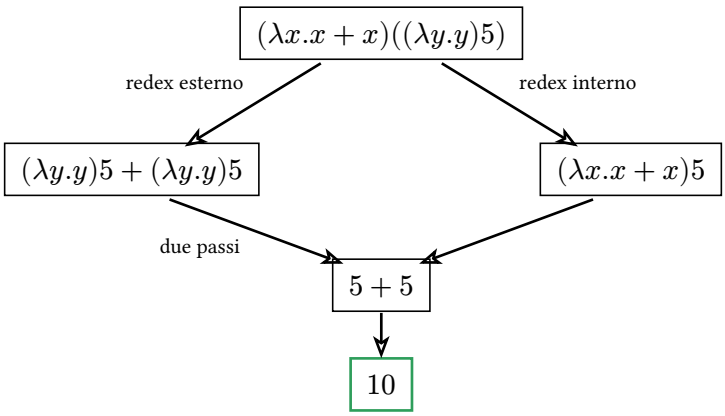
1.5 Ordine di riduzione e strategie

Quando un'espressione contiene più redex, si può scegliere da quale cominciare. In $(\lambda x.x)((\lambda y.y)z)$ ce ne sono due:



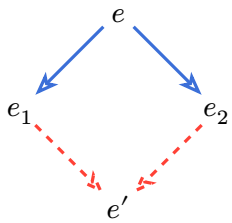
Strade diverse, stesso risultato

Ma in generale l'ordine di valutazione può cambiare il risultato finale? Un altro esempio (usando il $+$):



Scegliendo prima il redex esterno si copia $(\lambda y.y)5$ e lo si riduce due volte; prima quello interno, una volta sola. Il risultato è lo stesso

La risposta generale è il **teorema di Church-Rosser**: l'ordine in cui si scelgono le β-riduzioni **non influisce sul risultato finale**. Più precisamente: se a una stessa espressione si possono applicare due riduzioni (o sequenze di riduzioni) diverse, esiste un'espressione raggiungibile da entrambi i risultati con altre riduzioni (anche nessuna).



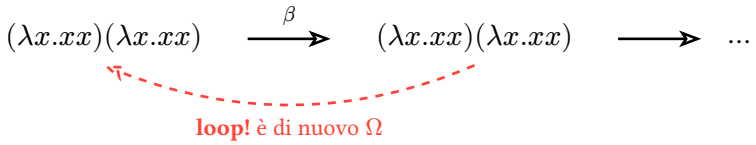
Proprietà di confluenza
(o del diamante)

Per questo nell'esempio sopra entrambe le strade arrivano a 10.

Non sempre però si arriva a una forma normale. Il combinatore

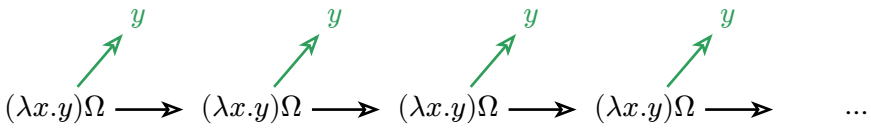
$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

contiene un solo redex, e ridurlo dà di nuovo Ω: la funzione $\lambda x.xx$ applica il suo argomento a se stesso, e l'argomento è proprio $\lambda x.xx$.



Ω non si può ridurre in forma normale: è un combinatore **divergente**, l'equivalente di un ciclo infinito.

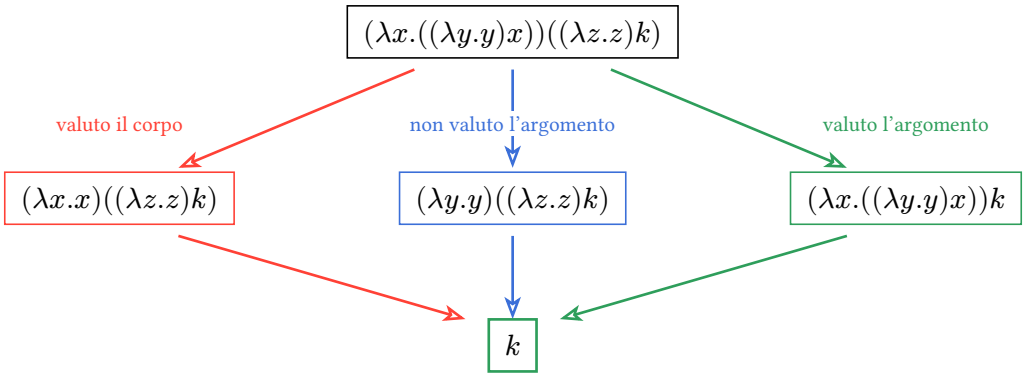
Una stessa espressione può **terminare** facendo certe scelte di riduzione e **non terminare** facendone altre. Per esempio $(\lambda x.y)\Omega$:



In orizzontale riduco dentro Ω e resto sempre fermo; in qualunque momento posso invece applicare $\lambda x.y$, che butta via l'argomento e dà y

Church-Rosser garantisce che **in tutti i casi in cui la riduzione termina, il risultato è lo stesso**: non possono esserci risultati diversi.

Le scelte possibili sono di tre tipi. In $(\lambda x.((\lambda y.y)x))((\lambda z.z)k)$:



Tre scelte diverse, stesso risultato k

Succede lo stesso in un linguaggio. Con `function f(x) { 3+2 + x }` e la chiamata `f(2+1)` posso calcolare l'argomento prima di passarlo (`f(3)`), passarlo non calcolato e lasciarlo alla funzione (`f(2+1)`), oppure semplificare il corpo (`5 + x`).

Una **strategia di valutazione** fissa quale scelta fare. Valutare un'applicazione $(\lambda x.e)e'$ ha tre passi: valuto l'espressione che definisce la funzione, passo il parametro, valuto il corpo. Le due strategie cambiano il secondo passo, il **passaggio dei parametri**:

	Call-by-value (CBV), *eager*	**Call-by-name (CBN), *lazy***
Cosa fa con l'argomento e'	lo valuta **prima** della β-riduzione	fa **subito** la β-riduzione
Al posto di x finisce	il **valore** dell'argomento	l'**espressione non valutata**
Redex scelto	il più interno	il più esterno
Linguaggi	OCaml e la maggior parte	Haskell

Le strategie si scrivono con **regole**: sopra la riga l'ipotesi, sotto la conclusione. La prima a sinistra dice: se e_1 fa un passo e diventa e', allora e_1e_2 diventa $e'e_2$.

Standard (senza strategia)

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\}$$

$$\frac{e_1 \to e'}{e_1e_2 \to e'e_2} \qquad \frac{e_2 \to e'}{e_1e_2 \to e_1e'}$$

$$\frac{e \to e'}{\lambda x.e \to \lambda x.e'}$$

Call-by-value

$$(\lambda x.e_1)v_2 \to e_1\{x := v_2\}$$

$$\frac{e_1 \to e'}{e_1e_2 \to e'e_2}$$

$$\frac{e_2 \to e'}{v_1e_2 \to v_1e'}$$

Call-by-name

$$(\lambda x.e_1)e_2 \to e_1\{x := e_2\}$$

$$\frac{e_1 \to e'}{e_1e_2 \to e'e_2}$$

- **Standard**: qualsiasi redex si può ridurre in qualsiasi momento, anche dentro il corpo di una funzione (terza riga). Non è deterministica.
- **Call-by-value**: v sta per un **valore**, cioè un'espressione già valutata. La β-riduzione scatta solo quando l'argomento è un valore v_2; l'argomento e_2 si riduce solo quando la funzione è già un valore v_1.
- **Call-by-name**: si riduce la parte funzione finché diventa $\lambda x.e_1$, poi si applica. L'argomento non si tocca.

> CBV e CBN **non hanno la regola che riduce il corpo di una funzione**. Quindi $\lambda x.((\lambda y.y)z)$ con la riduzione standard diventa $\lambda x.z$, con le due strategie resta così com'è: per loro una funzione è già un valore, anche se dentro ha un redex. È quello che succede nei linguaggi: il corpo di una funzione non viene eseguito finché la funzione non viene chiamata.

La stessa espressione con le due strategie (usando numeri e $+$):

Call-by-value: redex più interno

$$
\begin{aligned}
&(\lambda x.\lambda y.yx)(5+2)(\lambda x.x+1)\\
\to\ &(\lambda x.\lambda y.yx)7(\lambda x.x+1)\\
\to\ &(\lambda y.y7)(\lambda x.x+1)\\
\to\ &(\lambda x.x+1)7\\
\to\ &7+1\to 8
\end{aligned}
$$

Call-by-name: redex più esterno

$$
\begin{aligned}
&(\lambda x.\lambda y.yx)(5+2)(\lambda x.x+1)\\
\to\ &(\lambda y.y(5+2))(\lambda x.x+1)\\
\to\ &(\lambda x.x+1)(5+2)\\
\to\ &(5+2)+1\\
\to\ &7+1\to 8
\end{aligned}
$$

Conta quale strategia si sceglie?
- Se la valutazione termina, **no**: per la confluenza il risultato è lo stesso.
- La CBV valuta ogni argomento **una volta sola**. La CBN lo valuta **solo se serve**, ma può valutarlo più volte: è la strada di sinistra nella figura di $(\lambda x.x+x)((\lambda y.y)5)$.
- La CBN **termina ogni volta che è possibile**; la CBV può non terminare anche quando la CBN termina. In $(\lambda x.y)\Omega$ la CBN applica subito $\lambda x.y$ e dà y; la CBV deve prima valutare l'argomento Ω e non finisce mai. Non contraddice la confluenza, che dice che una strada verso il termine comune **esiste**, non quale strategia la trova.

1.6 Funzioni con più argomenti

Nel λ-calcolo ogni funzione prende **un solo argomento**. Una funzione di due argomenti si scrive come **catena** di funzioni di un argomento, come la selezione $\lambda x.\lambda y.x$: la prima prende x e restituisce una funzione che prende y.

$$\lambda x.(\lambda y.xy) \qquad \text{da}\ \ f : (X \times Y) \to Z\ \ \text{a}\ \ g : X \to (Y \to Z)$$

Questa trasformazione si chiama **currying** (dal logico Haskell Curry). g prende un X e restituisce una funzione da Y a Z, e calcola gli stessi risultati di f:

$$(\lambda x.\lambda y.x+y)10 \to \lambda y.10+y \qquad (\lambda y.10+y)5 = 15$$

$\lambda y.10+y$ è una funzione a sé: somma 10 a quello che le passi. Dare a una funzione solo il primo argomento si chiama **applicazione parziale**: ottieni una funzione che aspetta gli altri.

L'espressività non cambia, il risultato è lo stesso che avrei con due parametri insieme:

Definizione	**Chiamata**
$F = \lambda(x,y).e$	$F(e_1,e_2) \to e\{x := e_1\}\{y := e_2\}$
$F = \lambda x.\lambda y.e$	$(Fe_1)e_2 \to (\lambda y.e)\{x := e_1\}e_2 \to e\{x := e_1\}\{y := e_2\}$

In JavaScript, `curry` prende una funzione `f` che vuole due argomenti insieme e restituisce una funzione che li prende uno alla volta (nota le funzioni annidate):

```
function curry(f) {
  return function(a) {
    return function(b) {
      return f(a, b);
    };
  };
}
```

```
function sum(a, b) { return a + b; }

let curriedSum = curry(sum);
let h = curriedSum(1); // somma sempre 1
alert( h(2) );         // 3
alert( curriedSum(1)(2) ); // 3
```

`sum` passa da `Int × Int → Int` a `Int → (Int → Int)`. I vantaggi del currying:

- **uniformità**: ogni funzione ha un solo argomento, non serve una regola a parte per più argomenti;
- **applicazione parziale**: funzioni specializzate (come `h`) senza costrutti in più;
- **minimalità**: astrazione e applicazione bastano per funzioni con qualsiasi numero di argomenti.

Tutto questo si regge sull'**ordine superiore** (*higher order*). Altri due esempi: Twice prende una funzione e la applica due volte:

$$\text{Twice} = \lambda f.\lambda x.f(fx)$$

$$\text{Twice}(\lambda y.y+y)2 \Rightarrow (\lambda y.y+y)((\lambda y.y+y)2) \Rightarrow (\lambda y.y+y)4 \Rightarrow 8$$

Comp è la **composizione** $f \circ g$: prende due funzioni e ne restituisce una nuova che applica prima g e poi f. Ha tipo $(B \to C) \times (A \to B) \to (A \to C)$.

$$\text{Comp} = \lambda f.\lambda g.\lambda x.f(gx) \qquad x \to \boxed{g} \to gx \to \boxed{f} \to f(gx)$$

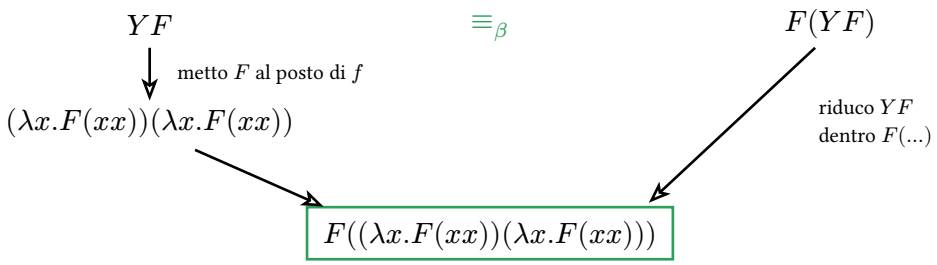
1.7 Ricorsione: il combinatore Y

Nel λ-calcolo non c'è un meccanismo per la ricorsione: le funzioni sono anonime, quindi una funzione non ha un nome con cui richiamare se stessa. Il problema si aggira con il **combinatore di punto fisso** Y.

Un **punto fisso** di una funzione F è un termine p tale che $F(p) = p$. Y è una funzione di ordine superiore che, data F, ne costruisce un punto fisso:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx)) \qquad YF \equiv_\beta F(YF)$$

Perché vale la proprietà: YF e $F(YF)$ si riducono allo stesso termine.



Nel secondo passo $\lambda x.F(xx)$ è passata a se stessa come argomento: ogni xx diventa una nuova copia del termine di partenza, con una F davanti. Continuando si ottiene $F(F(\ldots))$: una «copia infinita» della funzione.

Per definire una funzione ricorsiva $F = \langle\text{corpo che contiene } F\rangle$ si fanno due passi:

1. si scrive $G = \lambda f.\langle\text{espressione che contiene } f\rangle$: il nome della funzione diventa un parametro;
2. si definisce $F = YG$.

Allora $F = YG \equiv_\beta G(YG)$, cioè l'espressione con YG al posto di f; e dentro, di nuovo, $YG \equiv_\beta G(YG)$. Y «srotola» la definizione una chiamata alla volta, senza usare il nome della funzione. Il caso che ferma la ricorsione lo deve mettere il programmatore.

Esempio: fattoriale (supponendo di avere numeri, if, $n = 0$, $n - 1$, $*$).

$$G = \lambda f.\lambda n.\ \text{if}(n = 0)\ \text{then}\ 1\ \text{else}\ n * f(n - 1)$$

Calcolo $YG1$. Per brevità $W = (\lambda x.G(xx))(\lambda x.G(xx))$, il termine che si ricopia:

$$\begin{aligned}
YG1 &\to W1 \\
&\to GW1 \\
&\to (\lambda n.\ \text{if}(n = 0)\ \text{then}\ 1\ \text{else}\ n * W(n - 1))1 \\
&\to \text{if}(1 = 0)\ \text{then}\ 1\ \text{else}\ 1 * W(1 - 1) \\
&\overset{*}{\to} 1 * W0 \\
&\to 1 * GW0 \\
&\to 1 * (\lambda n.\ \text{if}(n = 0)\ \text{then}\ 1\ \text{else}\ n * W(n - 1))0 \\
&\to 1 * (\text{if}(0 = 0)\ \text{then}\ 1\ \text{else}\ 0 * W(0 - 1)) \\
&\overset{*}{\to} 1 * 1 = 1
\end{aligned}$$

Nell'ultimo if la condizione è vera: $W(0 - 1)$ non viene mai calcolato e la ricorsione si ferma.

> Y funziona con la **call-by-name**, che non valuta subito gli argomenti: in $F(xx)$ l'argomento xx viene ridotto solo se e quando serve nel corpo di F. Con la call-by-value xx verrebbe valutato subito, e sarebbe un loop infinito.

Per questo in JavaScript, che è call-by-value, Y è un po' diverso. `x(x)` diventa `y => x(x)(y)`: non è più un valore da calcolare subito ma una funzione che aspetta ancora l'argomento `y`, e la ricorsione parte solo quando `f` viene davvero chiamata.

```
const Y = f => (x => x(x))(x => f(y => x(x)(y)));
const factorial = f => (x => (x === 1 ? 1 : x * f(x - 1)));
const YFactorial = Y(factorial)(10);
```

1.8 Codifiche: booleani e numeri

Il λ-calcolo puro ha solo funzioni. Booleani, condizionali e numeri (quello che serve per essere Turing-equivalenti) si ottengono con le **codifiche**. L'idea: non guardare cosa un valore **rappresenta** ma cosa ci si può **fare**, e descrivere quell'uso con una funzione.

Con un **booleano** si fa una scelta fra due alternative. Quindi un booleano è una funzione che, date due scelte, ne seleziona una:

$\text{TRUE} = \lambda t.\lambda f.t$	$\text{TRUE}\,ab \Rightarrow a$
$\text{FALSE} = \lambda t.\lambda f.f$	$\text{FALSE}\,ab \Rightarrow b$

Da qui gli operatori logici: NOT applica il booleano b alle due scelte FALSE e TRUE, così TRUE sceglie FALSE e viceversa.

$$\text{NOT} = \lambda b.b\,\text{FALSE}\,\text{TRUE}$$

$$\text{NOT TRUE} \to \text{TRUE FALSE TRUE} = (\lambda t.\lambda f.t)\,\text{FALSE TRUE} \Rightarrow \text{FALSE}$$
$$\text{NOT FALSE} \to \text{FALSE FALSE TRUE} = (\lambda t.\lambda f.f)\,\text{FALSE TRUE} \Rightarrow \text{TRUE}$$

Con la stessa idea, AND e OR usano il primo booleano b per scegliere. AND: se b è vero il risultato è c, altrimenti FALSE. OR: se b è vero il risultato è TRUE, altrimenti c.

$$\text{AND} = \lambda b.\lambda c.bc\,\text{FALSE} \qquad \text{OR} = \lambda b.\lambda c.b\,\text{TRUE}\,c$$

$$\begin{aligned}\text{AND TRUE TRUE} &\Rightarrow \text{TRUE TRUE FALSE} = (\lambda t.\lambda f.t)\,\text{TRUE FALSE}\\ &\to (\lambda f.\,\text{TRUE})\,\text{FALSE} \to \text{TRUE}\\ \text{OR TRUE FALSE} &\Rightarrow \text{TRUE TRUE FALSE} = (\lambda t.\lambda f.t)\,\text{TRUE FALSE}\\ &\to (\lambda f.\,\text{TRUE})\,\text{FALSE} \to \text{TRUE}\end{aligned}$$

Il **condizionale** sceglie in base al valore della condizione: passa i due rami al booleano c.

$$\text{IF} = \lambda c.\lambda\ \text{then}.\lambda\ \text{else}.c\ \text{then else} \qquad \text{IF TRUE}\,ab \Rightarrow a \qquad \text{IF FALSE}\,ab \Rightarrow b$$

$$\begin{aligned}\text{IF TRUE}\,e_1e_2 &= (\lambda c.\lambda\ \text{then}.\lambda\ \text{else}.c\ \text{then else})\,\text{TRUE}\,e_1e_2\\ &\to (\lambda\ \text{then}.\lambda\ \text{else}.\,\text{TRUE then else})e_1e_2\\ &\to (\lambda\ \text{else}.\,\text{TRUE}\,e_1\ \text{else})e_2\\ &\to \text{TRUE}\,e_1e_2 = (\lambda t.\lambda f.t)e_1e_2\\ &\to (\lambda f.e_1)e_2 \to e_1\end{aligned}$$

Esercizio: calcolare IF FALSE e_1e_2.

I **numeri naturali** hanno una definizione induttiva: zero è un naturale; se n è un naturale, anche Succ n (il successore) lo è. Le operazioni si definiscono allo stesso modo:

	Caso zero	**Caso successore**
Somma	$m + 0 = m$	$m + \text{Succ}\ n = \text{Succ}(m + n)$
Prodotto	$m \times 0 = 0$	$m \times \text{Succ}\ n = m \times n + m$

Nel λ-calcolo un numero si codifica con **cosa ci si fa**: ripetere qualcosa n volte. Il **numerale di Church** C_n prende una funzione s (il successore) e un valore iniziale z (lo zero), e applica s a z per n volte. Il numero 3 vuol dire «fai una cosa tre volte».

$$C_0 = \lambda s.\lambda z.z \qquad C_1 = \lambda s.\lambda z.sz \qquad C_2 = \lambda s.\lambda z.s(sz)$$
$$C_3 = \lambda s.\lambda z.s(s(sz)) \qquad C_n = \lambda s.\lambda z.s^n z$$

Le operazioni sui numerali:

Definizione	**Come funziona**
$\text{SUCC} = \lambda n.\lambda s.\lambda z.s(nsz)$	nsz applica s a z per n volte; poi una s in più: in tutto $n+1$ volte. $\text{SUCC}\, C_1 \Rightarrow C_2$
$\text{PLUS} = \lambda m.\lambda n.\lambda s.\lambda z.ms(nsz)$ oppure $\lambda m.\lambda n.m\,\text{SUCC}\,n$	nsz calcola n; $ms(\ldots)$ ci applica s altre m volte. Nella seconda forma: applico SUCC a n per m volte. $\text{PLUS}\, C_i C_j \Rightarrow \text{SUCC}^i(C_j)$
$\text{TIMES} = \lambda m.\lambda n.m(\text{PLUS}\,n)C_0$	m conta le ripetizioni: partendo da C_0, sommo n per m volte. $\text{TIMES}\, C_i C_j \Rightarrow \text{PLUS}\, C_j(\text{PLUS}\, C_j(\ldots(\text{PLUS}\, C_j C_0)))$ con i somme

$\text{SUCC}\, C_1$ passo per passo. Per non confondere le variabili, in C_1 rinomino s, z in s', z'; il secondo passo riduce dentro il corpo della funzione (riduzione standard):

$$\begin{aligned}\text{SUCC}\, C_1 = (\lambda n.\lambda s.\lambda z.s(nsz))C_1 &\to \lambda s.\lambda z.s(C_1 sz) \\ &= \lambda s.\lambda z.s((\lambda s'.\lambda z'.s'z')sz) \\ &\Rightarrow \lambda s.\lambda z.s(sz) = C_2\end{aligned}$$

Esercizi:
1. definire ISZERO in modo che $\text{ISZERO}\, C_0 = \text{TRUE}$ e $\text{ISZERO}\, C_i = \text{FALSE}$ per $i \neq 0$; calcolare $\text{ISZERO}\, C_1$;
2. calcolare $\text{PLUS}\, C_1 C_1$.

Quindi il λ-calcolo è **Turing completo**: con codifiche furbe rappresenta quasi ogni costrutto dei linguaggi. Ma i programmi sarebbero lenti (migliaia di chiamate di funzione), lunghi (centinaia di righe) e difficili da capire. In pratica si usano linguaggi più ricchi, che hanno già le primitive giuste (numeri, booleani, `if`).

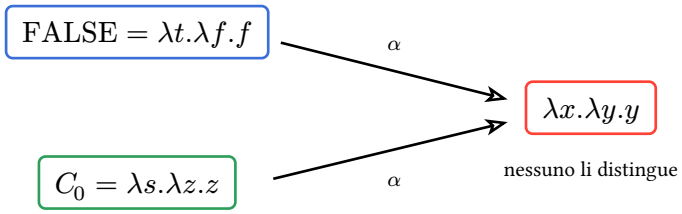
2 Controllo dei tipi

2.1 Perché servono i tipi

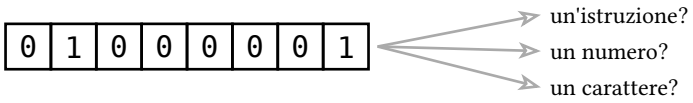
Nel λ-calcolo programmi e valori sono tutti funzioni. Quindi niente impedisce di usare un valore nel posto sbagliato:

$$\text{FALSE}\, C_0 = (\lambda t.\lambda f.f)(\lambda s.\lambda z.z) \to \lambda f.f$$

FALSE si può davvero applicare a C_0, e produce un valore. Ma non ha senso: FALSE serve a scegliere fra due alternative, non a ricevere un numero. Peggio: C_0 e FALSE sono **lo stesso termine**, a meno di α-conversione. Lo stesso vale per $\text{IF}\, C_0 e_1 e_2$ o $\text{TRUE}\, C_0$.



Non è solo un problema del λ-calcolo. Nel **linguaggio macchina** le istruzioni sono parole della macchina (sequenze di bit), i dati sono parole della macchina, e le operazioni prendono sequenze di bit e restituiscono sequenze di bit.



Un principio centrale dell'ingegneria del software è **trovare presto gli errori di programmazione** (come $\text{FALSE}\, C_0$). I linguaggi lo fanno con i **sistemi di tipo** (e più in generale con gli strumenti di **analisi statica**, che esaminano il programma senza eseguirlo).

I **tipi** raggruppano i valori in base al comportamento che vorremmo avessero. Il tipo di un dato dice come il linguaggio permette di usarlo. Un tipo:

- **limita i valori** che un'espressione (una variabile, una funzione) può assumere;
- **definisce le operazioni** ammesse su quei valori e come vengono memorizzati.

I linguaggi hanno dei tipi di base: interi (di varie dimensioni), numeri in virgola mobile (approssimano i reali), caratteri, booleani. $e : T$ si legge «e ha tipo T»:

Espressione	Cosa dice
$3 + 2 : \text{Int}$ ✓	il risultato è un intero, e $+$ è usato bene: riceve due interi
$\text{true} : \text{Int}$ ✗	true non è un intero
$2 + \text{true}$ ✗	$+$ non è applicato bene

Perché controllare i tipi? In un programma grande c'è questo pezzo:

```
if (condizione_complicata)
  { return "hello"/10; }
```

Se i test non rendono mai vera `condizione_complicata`, la divisione di una stringa nel ramo `then` non viene mai scoperta. Poi un input non previsto dai test fa eseguire il ramo, e il programma va in errore **durante l'esecuzione**. Bastava segnalare `"hello"/10` come espressione non ammessa.

Lo stesso codice in JavaScript e in TypeScript:

```
function addNumbers(x, y) {
  return x + y; }
console.log(addNumbers(3, "0"));
```

JavaScript stampa `"30"`, una stringa. Il `+` accetta anche una coppia (numero, stringa): il numero viene trasformato in stringa (**coercion**) e le due stringhe attaccate.

```
function addNumbers(x: number,
                    y: number) {
  return x + y; }
console.log(addNumbers(3, "0"));
```

TypeScript si accorge del bug e dà un errore di tipo: *Argument of type "string" is not assignable to parameter of type "number"*. Come? Con delle **regole di tipo**.

2.2 Sistemi di tipi

«Well-typed programs cannot go wrong» — Robin Milner

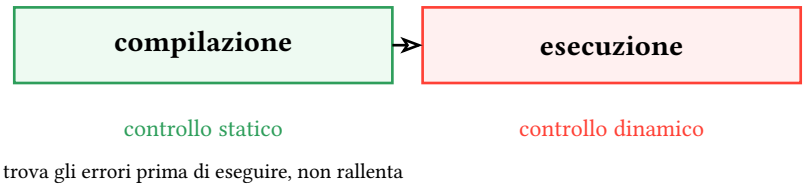
Un **sistema di tipi** associa tipi ai valori calcolati. Seguendo il flusso dei valori, prova a **dimostrare** che non avvengano errori di tipo. È il sistema stesso a stabilire cos'è un errore di tipo: un'operazione che si aspetta un certo tipo di valore usata con un valore per cui non ha senso.

Più precisamente è un metodo **sintattico**, **effettivo** e **strutturale** per dimostrare che il programma non ha comportamenti anomali:

Metodo	Cosa vuol dire
sintattico	l'analisi segue la struttura sintattica del programma
effettivo	c'è un algoritmo che controlla i vincoli sui tipi, da mettere in un compilatore o in un interprete
strutturale	il tipo di un'espressione dipende solo dai tipi delle sue sottoespressioni (**composizionale**)

Type safety (uso corretto dei tipi): se un'espressione ha un tipo, il tipo garantisce che venga usata nel modo giusto. Senza type safety si scrivono programmi pieni di bug, e i bug sono **vulnerabilità**: un attaccante può sfruttarli per cambiare il comportamento del programma o prenderne il controllo.

Il controllo può essere **statico**, **dinamico** o un misto dei due:



Il **type checker** (controllore dei tipi) verifica che il programma rispetti le intenzioni del programmatore, scritte nelle **annotazioni di tipo** (come `x: number`). Se un programma supera il controllo, durante l'esecuzione **non applicherà mai un'operazione a un valore del tipo sbagliato**. È una **correttezza parziale**: il programma è corretto rispetto ai tipi, non rispetto ad altri errori.

Resta una domanda: come si fa a sapere che i programmi che superano il controllo si comportano davvero bene durante l'esecuzione? Servono due cose:

1. un modo per **specificare** un sistema di tipi;
2. un modo per **dimostrare** che è corretto. Per questo bisogna capire nel dettaglio cosa succede quando un programma viene eseguito, cioè il suo **significato**.

Lo strumento per entrambe sono le regole di inferenza.

2.3 Regole di inferenza

Le abbiamo già usate per le strategie di valutazione: sopra la riga le **premesse**, sotto la **conclusione**, a destra il **nome** della regola. Se valgono tutte le premesse, si può concludere q. Una regola senza premesse è un **assioma**: q è un fatto accettato dal sistema.

$$\frac{p_1 \quad p_2 \quad \dots \quad p_n}{q}\ \text{(nome)} \qquad \frac{}{q}\ \text{(assioma)}$$

Una conclusione è valida **rispetto alle regole del sistema scelto**. Esempio: le regole per l'uguaglianza fra termini, riflessiva e transitiva:

$$\frac{}{t = t}\ \text{(Rifl)} \qquad \frac{t_1 = t_2 \quad t_2 = t_3}{t_1 = t_3}\ \text{(Trans)}$$

Una dimostrazione mette insieme più regole in un **albero di derivazione**: le foglie in alto (premesse o assiomi), ogni riga è una regola applicata, la radice in basso è la conclusione. Dalle premesse $a = b$, $b = c$, $c = d$ si deriva $a = d$:

$$\frac{\dfrac{a = b \quad b = c}{a = c}\ \text{(Trans)} \quad c = d}{a = d}\ \text{(Trans)}$$

Le regole possono anche **definire un insieme**: le regole generano gli elementi, una derivazione dimostra che un elemento ci sta dentro. Per esempio i pari:

$$\{0\} \subseteq \text{Pari} \Rightarrow \{0, 2\} \subseteq \text{Pari} \Rightarrow \{0, 2, 4\} \subseteq \text{Pari} \Rightarrow \dots$$

$$\text{Pari} = \{0, 2, 4, 6, \dots\}$$

$$\frac{}{0 \in \text{Pari}}\ \text{(Zero)} \qquad \frac{(n-2) \in \text{Pari}}{n \in \text{Pari}}\ \text{(Passo, } n \geq 2\text{)}$$

Derivazione di $4 \in \text{Pari}$:

$$\frac{\dfrac{\dfrac{}{0 \in \text{Pari}}\ \text{(Zero)}}{2 \in \text{Pari}}\ \text{(Passo)}}{4 \in \text{Pari}}\ \text{(Passo)}$$

Per dimostrare una proprietà di **tutti** gli elementi generati si segue la struttura delle regole: è l'**induzione sulle derivazioni**. Un caso per ogni regola: gli assiomi sono i casi base, le altre regole i passi induttivi.

Proprietà da dimostrare: $P(n) \equiv n \bmod 2 = 0$.

Caso base: regola (Zero)	**Passo induttivo: regola (Passo)**
$P(0)$ vale perché $0 \bmod 2 = 0$.	Suppongo $P(n-2)$ e dimostro $P(n)$. $(n-2) \bmod 2 = 0$ vuol dire che esiste k con $n - 2 = 2k$. Allora $n = 2k + 2 = 2(k+1)$: con $k' = k + 1$ ho $n = 2k'$, quindi $n \bmod 2 = 0$.

Conclusione: per ogni $n \in \text{Pari}$ vale $n \bmod 2 = 0$.

2.4 Un linguaggio di espressioni

Primo caso di studio: un sistema di tipi per un piccolo linguaggio di espressioni con booleani e numeri. Prima la **sintassi**:

Espressioni

$$
\begin{aligned}
E ::= \ & \text{true} \\
\mid \ & \text{false} \\
\mid \ & \text{NV} \\
\mid \ & \text{if } E \text{ then } E \text{ else } E \\
\mid \ & \text{succ } E \\
\mid \ & \text{pred } E \\
\mid \ & \text{isZero } E
\end{aligned}
$$

Valori

$$
\begin{aligned}
V ::= \ & \text{true} \\
\mid \ & \text{false} \\
\mid \ & \text{NV}
\end{aligned}
$$

Valori numerici

$$\text{NV} ::= 0 \mid 1 \mid 2 \mid \ldots$$

I **valori** sono i risultati finali: le espressioni che non vanno più calcolate.

Poi la **semantica**, cioè cosa succede quando un'espressione viene eseguita: un **interprete** scritto con regole di inferenza. Ogni regola dice come fare **un passo** di calcolo $E \to E'$ (semantica *small-step*, a piccoli passi).

Condizionale

$$\text{if true then } E_1 \text{ else } E_2 \to E_1 \quad \text{(IF-TRUE)} \qquad \text{if false then } E_1 \text{ else } E_2 \to E_2 \quad \text{(IF-FALSE)}$$

$$\frac{E \to E'}{\text{if } E \text{ then } E_1 \text{ else } E_2 \to \text{if } E' \text{ then } E_1 \text{ else } E_2} \quad \text{(IF-COND)}$$

IF-COND ha la stessa forma delle regole delle strategie: finché la condizione non è un valore, si calcola la condizione. Quando diventa true o false scatta IF-TRUE o IF-FALSE.

Operazioni sui numeri

$$\frac{E \to E'}{\text{succ } E \to \text{succ } E'} \qquad \frac{m = n + 1}{\text{succ } n \to m}$$

$$\frac{E \to E'}{\text{pred } E \to \text{pred } E'} \qquad \text{pred } 0 \to 0 \qquad \frac{n > 0 \quad m = n - 1}{\text{pred } n \to m}$$

$$\frac{E \to E'}{\text{isZero } E \to \text{isZero } E'} \qquad \text{isZero } 0 \to \text{true} \qquad \frac{n > 0}{\text{isZero } n \to \text{false}}$$

Lo schema è sempre lo stesso: la regola a sinistra calcola l'argomento finché diventa un numero n, le altre fanno l'operazione. pred 0 dà 0: non ci sono numeri negativi.

Passo dopo passo, `if isZero 0 then 2 else pred 1` si calcola così:

$$\frac{\text{isZero } 0 \to \text{true}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1 \to \text{if true then } 2 \text{ else pred } 1} \quad \text{(IF-COND)}$$

$$\text{if true then } 2 \text{ else pred } 1 \to 2 \quad \text{(IF-TRUE)}$$

Finora l'interprete sa solo calcolare. Ora gli aggiungiamo i **tipi**: solo due, Bool e Nat (i numeri).

$$T ::= \text{Bool} \mid \text{Nat}$$

Il controllo dei tipi è una relazione fra espressioni e tipi, scritta $E : T$ («E ha tipo T»). È definita da regole di inferenza, una per ogni forma sintattica: il metodo è **sintattico**, per induzione sulla struttura del programma.

Regole di tipo

$$\frac{}{\text{true} : \text{Bool}}\ \text{(T-TRUE)} \qquad \frac{}{\text{false} : \text{Bool}}\ \text{(T-FALSE)} \qquad \frac{}{n : \text{Nat}}\ \text{(T-NAT)}$$

$$\frac{E : \text{Bool} \quad E_1 : T \quad E_2 : T}{\text{if } E \text{ then } E_1 \text{ else } E_2 : T}\ \text{(T-IF)}$$

$$\frac{E : \text{Nat}}{\text{succ } E : \text{Nat}}\ \text{(T-SUCC)} \qquad \frac{E : \text{Nat}}{\text{pred } E : \text{Nat}}\ \text{(T-PRED)} \qquad \frac{E : \text{Nat}}{\text{isZero } E : \text{Bool}}\ \text{(T-ISZERO)}$$

La relazione di tipo è la **più piccola** relazione che soddisfa tutte le istanze di queste regole. Un'espressione E è **ben tipata** (*well-typed*) se esiste un T con $E : T$; altrimenti è *ill-typed*.

Controllo («E ha tipo T?»): dati E e T si cerca un **albero di derivazione** fatto di istanze delle regole. Per esempio:

$$\frac{\dfrac{}{0 : \text{Nat}}\ \text{(T-NAT)} \quad \dfrac{0 : \text{Nat}}{\text{isZero } 0 : \text{Bool}}\ \text{(T-ISZERO)} \quad \dfrac{}{2 : \text{Nat}}\ \text{(T-NAT)} \quad \dfrac{1 : \text{Nat}}{\text{pred } 1 : \text{Nat}}\ \text{(T-PRED)}}{\text{if isZero } 0 \text{ then } 2 \text{ else pred } 1 : \text{Nat}}\ \text{(T-IF)}$$

Inferenza: è il problema opposto, data solo E si cerca un T (e la derivazione).

Le regole sono **deterministiche**: per ogni forma sintattica ce n'è una sola, quindi si trasformano subito in un algoritmo che calcola il tipo guardando i tipi delle sottoespressioni:

```
typeof(E) =
  E = true | false          →  Bool
  E = n                     →  Nat
  E = if E1 then E2 else E3
      →  T1 = typeof(E1), T2 = typeof(E2), T3 = typeof(E3);
         se T1 = Bool e T2 = T3 allora T2, altrimenti errore
  E = succ E1 | pred E1     →  se typeof(E1) = Nat allora Nat, altrimenti errore
  E = isZero E1             →  se typeof(E1) = Nat allora Bool, altrimenti errore
```

Il tipo di un'espressione dipende solo dai tipi delle sue parti: è la **composizionalità**. Per questo T-IF vuole i due rami dello **stesso** tipo T, che diventa il tipo di tutto l'`if`: chi usa quell'`if` deve sapere un tipo preciso. Per esempio l'argomento di pred deve essere Nat e la condizione di un `if` deve essere Bool.

> I sistemi di tipo sono **imprecisi**: danno un'approssimazione prudente che considera tutte le esecuzioni possibili, e anche qualcuna impossibile. if true then 0 else false dà sicuramente un numero, ma non ha tipo: T-IF considera anche il ramo `else`, che a runtime non verrebbe mai eseguito, e Nat $\neq$ Bool.

2.5 Type safety

Un sistema di tipi è **corretto** (*type safe*, *sound*) se vale ciò che promette Milner: i programmi ben tipati non «vanno storti». Si traduce in due proprietà, da dimostrare entrambe:

Proprietà	Cosa dice
Progresso: se $E : T$ allora E è un valore oppure $E \to E'$ per qualche E'	un'espressione ben tipata **non si blocca** durante l'esecuzione: o ha finito, o può fare un passo
Conservazione: se $E : T$ e $E \to E'$ allora $E' : T$	le regole di esecuzione **preservano il tipo**: dopo un passo l'espressione resta ben tipata, con lo stesso tipo

Insieme garantiscono che l'esecuzione di un'espressione ben tipata non si blocca mai su un tipo sbagliato.

> Senza tipi può succedere che l'espressione non sia un valore e non esista nessuna regola per proseguire: if 0 then 2 else 1 resta bloccata, perché la condizione non è né true né false. Il progresso dice che con i tipi non capita.

Per la conservazione, esempio sull'albero visto prima: if isZero 0 then 2 else pred 1 : Nat fa un passo e diventa if true then 2 else pred 1, che ha ancora tipo Nat (T-TRUE, 2 : Nat e pred 1 : Nat danno T-IF). Poi diventa 2 : Nat.

Le dimostrazioni sono per **induzione sulla derivazione di $E : T$**: un caso per ogni regola di tipo. Si mostra il caso dell'`if` (gli altri sono analoghi).

Progresso. Casi base (T-TRUE, T-FALSE, T-NAT): E è già un valore. Caso $E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3$: per T-IF vale $E_1 : \text{Bool}$. Per ipotesi induttiva (a) E_1 è un valore, quindi true o false, e scatta IF-TRUE o IF-FALSE; oppure (b) $E_1 \to E_{1'}$, e scatta IF-COND. In ogni caso E fa un passo.

Conservazione. Casi base: E è un valore, non si riduce, la tesi è vera. Caso $E = \text{if } E_1 \text{ then } E_2 \text{ else } E_3 : T$: per T-IF vale $E_1 : \text{Bool}$, $E_2 : T$, $E_3 : T$. Il passo $E \to E'$ può venire da tre regole:

1. IF-TRUE: $E' = E_2$, che ha tipo T;
2. IF-FALSE: $E' = E_3$, che ha tipo T;
3. IF-COND con $E_1 \to E_{1'}$: per ipotesi induttiva $E_{1'} : \text{Bool}$, quindi con T-IF anche $\text{if } E_{1'} \text{ then } E_2 \text{ else } E_3 : T$.

3 Il lambda calcolo tipato

3.1 Tipi e regole di tipo

Il λ-calcolo «puro» non ha tipi. Qui lo rendiamo tipato, partendo da un solo tipo di base, i booleani, più il tipo delle funzioni:

$$\tau ::= \text{Bool} \mid \tau \to \tau$$

Esempi di tipi: Bool, Bool → Bool, (Bool → Bool) → Bool. La freccia **associa a destra**: Bool → Bool → Bool sta per Bool → (Bool → Bool). Un tipo $\tau_1 \to \tau_2$ è quello delle funzioni che prendono un argomento di tipo τ_1 e restituiscono un risultato di tipo τ_2.

Il linguaggio, che chiamiamo **FUN**, ha il parametro con il suo tipo (nel formato tradizionale: $\lambda x : \tau . e$):

Espressioni

$e ::= x$ variabile
| $\text{fun}\, x : \tau = e$ funzione
| $\text{Apply}(e, e)$ applicazione
| $\text{true} \mid \text{false}$
| $\text{if}\, e\, \text{then}\, e\, \text{else}\, e$

Valori

$v ::= \text{fun}\, x : \tau = e$
| $\text{true} \mid \text{false}$

Il condizionale è un'**espressione**, non un comando: i due rami contengono espressioni.

La semantica è quella già vista, riscritta con la nuova sintassi (sostituzione che evita la cattura). Per l'applicazione si usa **call-by-value**: prima si calcola la funzione, poi l'argomento, poi si fa la β-riduzione.

Semantica

$$\text{Apply}(\text{fun}\, x : \tau = e_1, v_2) \to e_1\{x := v_2\} \quad \text{(β-riduzione)}$$

$$\frac{e_1 \to e_{1'}}{\text{Apply}(e_1, e_2) \to \text{Apply}(e_{1'}, e_2)} \qquad \frac{e_2 \to e_{2'}}{\text{Apply}(v_1, e_2) \to \text{Apply}(v_1, e_{2'})}$$

$$\frac{e \to e'}{\text{if}\, e\, \text{then}\, e_1\, \text{else}\, e_2 \to \text{if}\, e'\, \text{then}\, e_1\, \text{else}\, e_2}$$

$$\text{if true then } e_1 \text{ else } e_2 \to e_1$$

$$\text{if false then } e_1 \text{ else } e_2 \to e_2$$

Il problema di tipare le funzioni. Per dare un tipo a $\text{fun}\, x : \tau_1 = e$ ci aspettiamo $\tau_1 \to \tau_2$, dove τ_2 è il tipo del corpo e. Ma il tipo di e dipende dal tipo di x, che si decide nella funzione. Serve ricordare i tipi delle variabili: l'**ambiente dei tipi** Γ, una funzione che associa nomi a tipi.

$$\Gamma = x_1 : \tau_1, x_2 : \tau_2, \ldots, x_k : \tau_k \quad \text{significa} \quad \Gamma(x_i) = \tau_i$$

$\Gamma, x : \tau$ è Γ estesa con $x : \tau$: vale τ per x, e $\Gamma(y)$ per ogni $y \neq x$. L'ambiente vuoto $\varnothing$ non lega nessuna variabile. Esempio con $\Gamma = x : \tau, y : \tau'$:

$\Gamma(x)$	$\Gamma(z)$	$(\Gamma, z : \tau_2)(z)$
τ	indefinito	τ_2

Il controllo dei tipi diventa una relazione a **tre posti**, il **giudizio di tipo**:

$$\Gamma \vdash e : \tau$$

«nell'ambiente Γ, l'espressione e ha tipo τ». Nel linguaggio dei compilatori l'ambiente dei tipi è la **tabella dei simboli** (*symbol table*); le regole le applica il compilatore durante l'analisi statica.

Regole di tipo di FUN

$$\frac{}{\Gamma \vdash \text{true} : \text{Bool}}\ \text{(T-TRUE)} \qquad \frac{}{\Gamma \vdash \text{false} : \text{Bool}}\ \text{(T-FALSE)} \qquad \frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}\ \text{(T-VAR)}$$

$$\frac{\Gamma \vdash e : \text{Bool} \quad \Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau}\ \text{(T-IF)}$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \text{fun } x : \tau_1 = e : \tau_1 \to \tau_2}\ \text{(T-FUN)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash \text{Apply}(e_1, e_2) : \tau_2}\ \text{(T-APPLY)}$$

- **T-VAR**: una variabile ha il tipo che le dà l'ambiente.
- **T-IF**: i rami hanno lo stesso tipo τ, che può essere anche un tipo funzione: si possono avere condizionali i cui rami sono funzioni.
- **T-FUN**: il parametro x è dichiarato con il suo tipo τ_1; per tipare il corpo si usa $\Gamma, x : \tau_1$. Questa dichiarazione **sovrascrive** eventuali dichiarazioni precedenti di x e vale solo nel corpo (è il suo *scope*). Il risultato è il tipo del corpo.
- **T-APPLY**: se e_1 trasforma argomenti di tipo τ_1 in risultati di tipo τ_2 e e_2 ha tipo τ_1, il risultato è di tipo τ_2.

Esempio: $\varnothing \vdash \text{fun } x : \text{Bool} = x : \text{Bool} \to \text{Bool}$, perché $\varnothing, x : \text{Bool} \vdash x : \text{Bool}$.

Esempio più grande: $\lambda f.f(\text{if false then true else false})$, cioè $\text{fun } f : \text{Bool} \to \text{Bool} = \text{Apply}(f, e_0)$ con $e_0 = \text{if false then true else false}$. Sia $\Gamma_f = \varnothing, f : \text{Bool} \to \text{Bool}$.

$$\dfrac{\dfrac{\dfrac{\Gamma_f(f) = B \to B}{\Gamma_f \vdash f : B \to B}\ \text{(T-VAR)} \quad \dfrac{\Gamma_f \vdash \text{false} : B \quad \Gamma_f \vdash \text{true} : B \quad \Gamma_f \vdash \text{false} : B}{\Gamma_f \vdash e_0 : B}\ \text{(T-IF)}}{\Gamma_f \vdash \text{Apply}(f, e_0) : B}\ \text{(T-APPLY)}}{\varnothing \vdash \text{fun } f : B \to B = \text{Apply}(f, e_0) : (B \to B) \to B}\ \text{(T-FUN)}$$

3.2 Type safety nel λ-calcolo tipato

Le due proprietà valgono anche qui, con una condizione sul progresso: l'espressione deve essere **chiusa** (nessuna variabile libera), cioè tipata nell'ambiente vuoto.

Proprietà	**Cosa dice**
Progresso: se $\varnothing \vdash e : \tau$ allora e è un valore oppure $e \to e'$	un'espressione chiusa e ben tipata non si blocca
Conservazione: se $\Gamma \vdash e : \tau$ e $e \to e'$ allora $\Gamma \vdash e' : \tau$	un passo di esecuzione preserva il tipo

Si dimostrano ancora per induzione sulle derivazioni di tipo.

Progresso. Costanti booleane: come prima. Variabili: caso banale, perché nell'ambiente vuoto una variabile non ha tipo. Astrazione: è già un valore. Applicazione $e = \text{Apply}(e_1, e_2)$ con $\varnothing \vdash e_1 : \tau_1 \to \tau_2$ e $\varnothing \vdash e_2 : \tau_1$: per ipotesi induttiva e_1 e e_2 sono valori o possono fare un passo.
- Se e_1 fa un passo, e fa un passo (prima regola dell'applicazione).
- Se e_1 è un valore e e_2 fa un passo, e fa un passo (seconda regola).
- Se sono entrambi valori, e_1 ha tipo funzione, quindi è $\text{fun } x : \tau_1 = e_3$, e e fa un passo con la β-riduzione.

Conservazione. Il caso difficile è l'applicazione $e = \text{Apply}(\text{fun } x : \tau_1 = e_3, e_2)$ con e_2 valore, che si riduce a $e' = e_3\{x := e_2\}$. Dal tipo di e si ricava $\Gamma, x : \tau_1 \vdash e_3 : \tau_2$ e $\Gamma \vdash e_2 : \tau_1$, e bisogna dimostrare $\Gamma \vdash$

$e_3\{x := e_2\} : \tau_2$. Ma abbiamo tipato e_3 e non $e_3\{x := e_2\}$: ci vuole un lemma che dica che la sostituzione non cambia il tipo.

> **Lemma di sostituzione**: se $\Gamma, x : \tau_1 \vdash e : \tau$ e $\Gamma \vdash e_1 : \tau_1$ allora $\Gamma \vdash e\{x := e_1\} : \tau$. Sostituire un parametro formale con un parametro attuale dello **stesso tipo** conserva il tipo. Esempio: se $\text{if } x \text{ then } e_1 \text{ else } e_2 : \text{Bool}$ nell'ipotesi $x : \text{Bool}$, allora anche $(\text{if } x \text{ then } e_1 \text{ else } e_2)\{x := \text{true}\} : \text{Bool}$. Si dimostra per induzione sulla derivazione di $\Gamma, x : \tau_1 \vdash e : \tau$; i casi interessanti sono variabili e astrazioni.

3.3 Estensioni e ricorsione

Si può estendere FUN per farlo diventare un vero linguaggio di programmazione. Con le costanti numeriche n (tipo Nat) e le operazioni binarie $e_1 \oplus e_2$ (con $\oplus \in \{+, -, *, /, \%\}$, di tipo noto $\tau_1 \times \tau_2 \to \tau$):

$$\frac{}{\Gamma \vdash n : \text{Nat}} \text{ (T-NUM)} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2 \quad \oplus : \tau_1 \times \tau_2 \to \tau}{\Gamma \vdash e_1 \oplus e_2 : \tau} \text{ (T-OP)}$$

Si aggiungono anche le **dichiarazioni locali** $\text{let } x = e_1 \text{ in } e_2$. Si calcola prima e_1; quando è un valore v lo si sostituisce a x in e_2. Il tipo di x è quello di e_1, e vale solo in e_2:

Dichiarazioni locali

$$\frac{e_1 \to e_{1'}}{\text{let } x = e_1 \text{ in } e_2 \to \text{let } x = e_{1'} \text{ in } e_2} \qquad \text{let } x = v \text{ in } e_2 \to e_2\{x := v\}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \text{ (T-LET)}$$

Ricorsione. Nel λ-calcolo tipato semplice tutti i programmi tipabili **terminano**. Quindi $\Omega = (\lambda x.xx)(\lambda x.xx)$ e il combinatore Y **non sono tipabili**. Il motivo è l'auto-applicazione xx:

$$\frac{\Gamma \vdash x : \tau_1 \to \tau_2 \quad \Gamma \vdash x : \tau_1}{\Gamma \vdash \text{Apply}(x, x) : \tau_2} \text{ (T-APPLY)}$$

x ha un solo tipo, quindi servirebbe $\tau_1 = \tau_1 \to \tau_2$, che è impossibile. Per avere la ricorsione bisogna aggiungere al linguaggio un costrutto apposta, tipabile e con le proprietà che vogliamo.

Si parte da un esempio. Il fattoriale è il **punto fisso** della funzione G, che prende l'approssimazione f del fattoriale e ne costruisce una migliore:

$$G = \text{fun } f : \text{Nat} \to \text{Nat} = \text{fun } n : \text{Nat} = \text{if isZero } n \text{ then } 1 \text{ else } n * f(\text{pred } n)$$

$$G : (\text{Nat} \to \text{Nat}) \to \text{Nat} \to \text{Nat}$$

Il costrutto $\text{fix } e$ calcola questo punto fisso, e si scrive $\text{fact} = \text{fix } G$.

Il costrutto fix

$$\frac{e \to e'}{\text{fix } e \to \text{fix } e'} \qquad \text{fix}(\text{fun } f : \tau = e) \to e\{f := \text{fix}(\text{fun } f : \tau = e)\}$$

$$\frac{\Gamma \vdash e : \tau \to \tau}{\Gamma \vdash \text{fix } e : \tau} \text{ (T-FIX)}$$

La regola di valutazione «srotola» una volta la ricorsione: il corpo e viene eseguito con f sostituita da fix di sé stesso, così alla prossima chiamata ricorsiva si srotola ancora. Per $\text{fact } 3 = (\text{fix } G)3$:

$$(\text{fix}\, G)3 \to (\text{fun}\, n : \text{Nat} = \text{if isZero}\, n \,\text{then}\, 1 \,\text{else}\, n * (\text{fix}\, G)(\text{pred}\, n))3 \xrightarrow{*} 3 * (\text{fix}\, G)(\text{pred}\, 3)$$

Il λ-calcolo tipato semplice con fix si chiama **PCF** (*Programming Computable Functions*), un linguaggio funzionale tipato introdotto da Gordon Plotkin nel 1977.

Per scrivere meno c'è lo zucchero sintattico $\text{let rec}\, x : \tau = e \,\text{in}\, e'$, che sta per $\text{let}\, x = \text{fix}(\text{fun}\, x : \tau = e) \,\text{in}\, e'$. In OCaml ci sono due primitive: `fun` per le funzioni non ricorsive e `let rec` per quelle ricorsive.

```
let plusOne = fun x -> x + 1;;
let rec fact x = if x <= 1 then 1 else x * fact (x - 1)
```

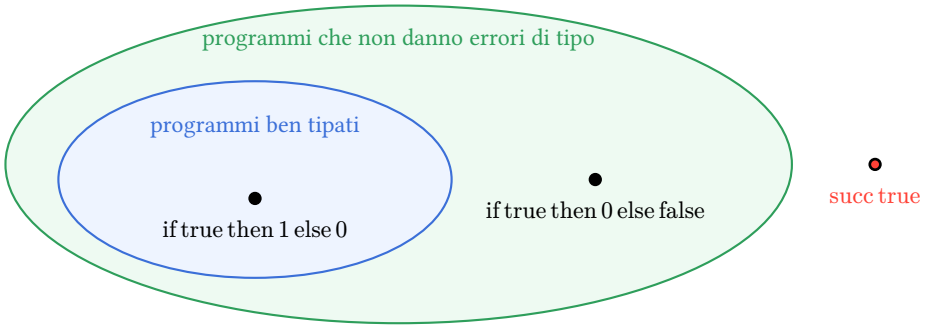
3.4 Cosa possono garantire i tipi

Con PCF il percorso sui tipi è completo, e resta da capire a cosa serve. Il λ-calcolo è un **modello in piccolo** dei linguaggi: sistemi di tipi, terminazione e verifica si studiano qui, dove le regole sono poche, e poi si portano ai linguaggi veri.

Un tipo è una **specifica di comportamento**: $\tau_1 \to \tau_2$ dice che la funzione prende un τ_1 e restituisce un τ_2, e il type checking controlla che il programma rispetti questa specifica. Con tipi più ricchi si controllano anche altre proprietà, per esempio la segretezza e l'autenticità nei protocolli di sicurezza, o l'assenza di deadlock (processi bloccati ad aspettarsi a vicenda) nei sistemi concorrenti.

La garanzia però è solo **parziale**, per un limite che non si può aggirare. Le domande interessanti su un programma (termina su tutti gli input? quanto può crescere la memoria dinamica? lascia uscire informazioni private?) riguardano cosa fa quando gira, e il **teorema di Rice** (1953) dice che **nessuna proprietà non banale del comportamento di un programma è decidibile**: non esiste un algoritmo che risponda sempre giusto («non banale» vuol dire vera per alcuni programmi e falsa per altri). Non basta nemmeno eseguire il programma e guardare, perché non c'è un limite al tempo che può impiegare.

Si esce dal problema **approssimando**: al posto della proprietà esatta se ne controlla una più prudente, che è decidibile. Un sistema di tipi fa proprio questo, e l'approssimazione deve rispettare i comportamenti veri: valgono progresso e conservazione, quindi un programma accettato non dà mai errori di tipo. Il prezzo è che qualche programma corretto viene rifiutato. Nel disegno ogni ovale è un insieme di programmi: i ben tipati stanno tutti dentro quelli senza errori di tipo (è la type safety), ma non li riempiono.



Il punto rosso sta fuori da tutto: succ true si blocca, e il type checker lo rifiuta giustamente. if true then 0 else false invece non dà errori, ma viene rifiutato lo stesso.

Su questa idea sono costruiti gli **analizzatori statici** usati nell'industria, che controllano il codice senza eseguirlo: per esempio Infer, FlowDroid e Roslyn Code Analyzer.